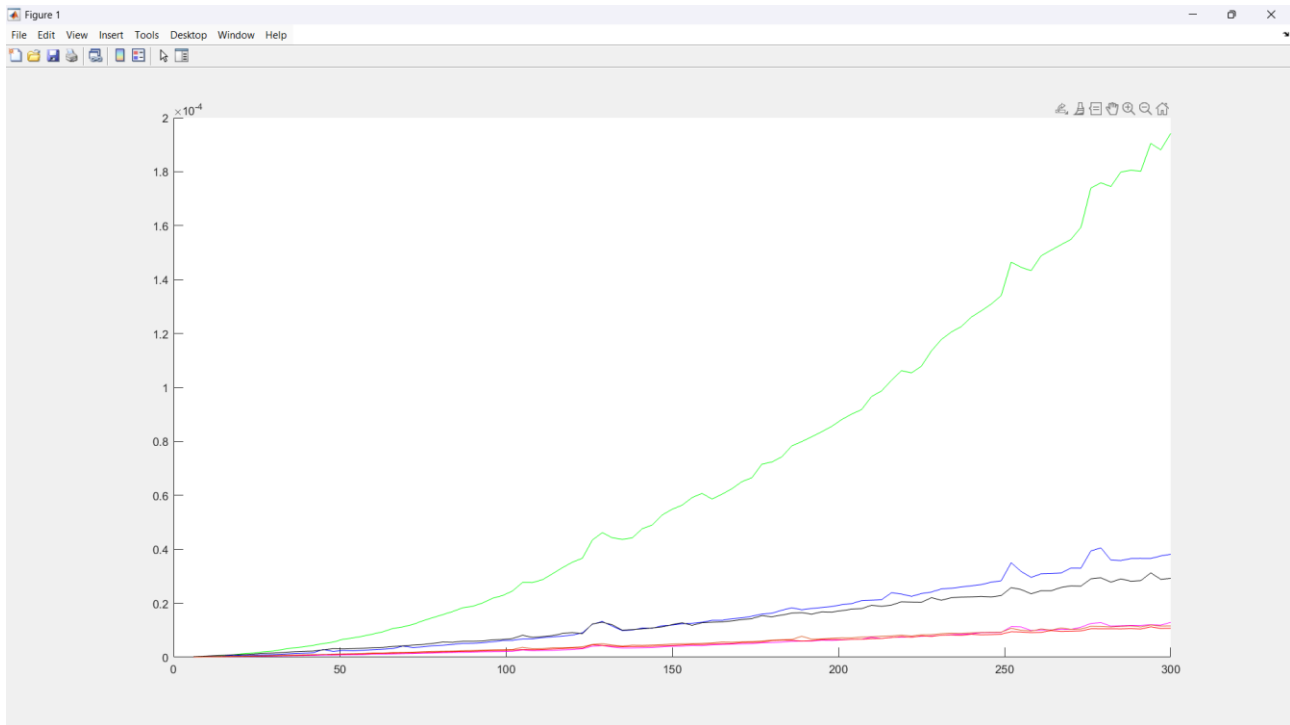


Tempi measure sorts

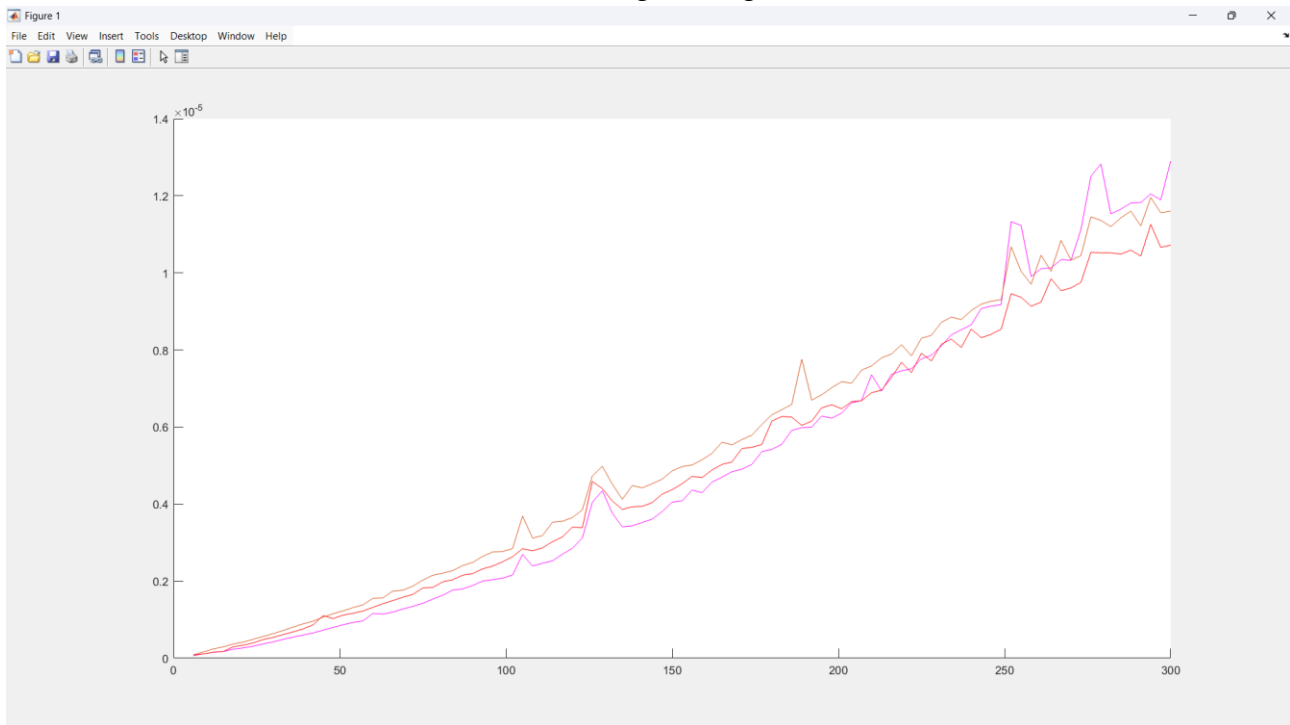
Il tempo medio per ordinare 300 vettori è riportato in funzione della loro dimensione nel seguente grafico.



Legenda:

- insertion sort: magenta
- quicksort: arancione
- `std::sort()`: rosso
- bubblesort: verde
- selection sort: blu
- mergesort: nero

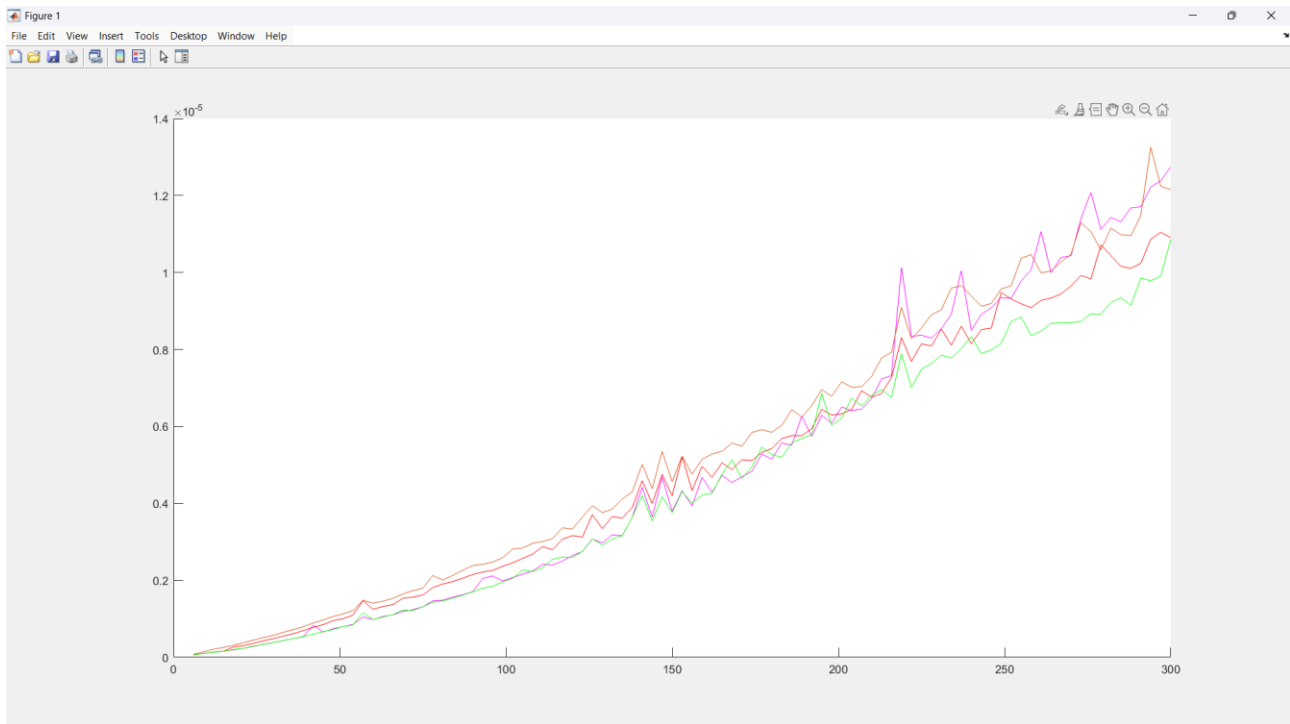
per visualizzare le differenze con maggiore precisione, possiamo eliminare il mergesort, il bubblesort e il selection sort. Otteniamo il seguente grafico:



osserviamo che il quicksort (arancione) diventa più efficiente dell'insertion sort (magenta) solo a partire dalla dimensione 215. Inoltre, lo `std::sort()` è meno efficiente dell'insertion sort per vettori di lunghezza circa <200 .

Si consideri che, rispetto all'esercitazione 4, i tempi di esecuzione dell'insertion sort sono più brevi, poiché l'ho implementato passando il vettore per riferimento e l'ho compilato in modalità release.

Alla luce di questi risultati, ho costruito il programma di ordinamento "personalizzato" (`mix_sort`) utilizzando l'insertion sort fino a vettori di lunghezza 215 e il quicksort successivamente. Ordinando vettori di lunghezza fino a 300 in modalità release, ho ottenuto il seguente grafico:



Legenda:

- insertion sort: magenta
- quicksort: arancione
- std::sort(): rosso
- mix_sort: verde

osserviamo che il mix sort impiega lo stesso tempo dell'insertion sort fino a vettori di lunghezza 215. Successivamente, diventa più veloce dello std::sort() ed anche del quicksort stesso. Quest'ultimo risultato è piuttosto inatteso, poiché il mix_sort utilizza il quicksort stesso e, inoltre, controlla la dimensione del vettore, dunque compie il numero di operazioni del quicksort +1. Il risultato non è imputabile all'inesattezza della costruzione del mix_sort, poiché è stato testato mediante la funzione is.sorted() presente in ordinamento.h.

USO DELL'AI

Ho riscontrato la presenza di termini negativi nei tempi misurati. Dopo aver controllato varie volte il codice, ho chiesto a Gemini quale potesse essere il problema e mi ha fatto sostituire, all'interno di timecounter.h e timecounter.cpp, high_resolution_clock con steady_clock. Con questa modifica non ho riscontrato più termini negativi nella misurazione dei tempi.